

Automated Medical Podcast Transcription and Topic Segmentation

Introduction

Medical podcasts contain valuable clinical insights, discussions, and expert opinions. However, extracting useful information from long audio files is time-consuming. This project provides an end-to-end automated solution that converts medical podcast audio into structured, readable, and searchable text.

The system performs audio preprocessing, speech-to-text transcription, segmentation, keyword extraction, summarization, and keyword frequency visualization.

Objectives

The main objectives of this project are:

- To automate transcription of medical podcast audio files
 - To segment long audio into meaningful sections
 - To extract important medical keywords
 - To generate concise and readable summaries
 - To visualize keyword frequency using word clouds
 - To provide a simple and interactive user interface
-

System Approach

The project follows a modular pipeline-based approach:

1. Audio uploaded by the user
 2. Audio preprocessing such as noise reduction and normalization
 3. Speech-to-text transcription using the Whisper model
 4. Segmentation of transcripts into logical parts
 5. Keyword extraction using TF-IDF techniques
 6. Summarization of each segment
 7. Keyword frequency visualization
 8. User interaction through a Streamlit-based interface
-

Tech Stack

Category	Technologies Used
Programming Language	Python 3.x
Speech-to-Text	OpenAI Whisper
Audio Processing	Pydub, FFmpeg
Natural Language Processing	NLTK, Scikit-learn
Visualization	Matplotlib, WordCloud
Frontend	Streamlit
Testing	Pytest
Version Control	Git

Setup and Installation

Prerequisites

- Python 3.9 or above
- FFmpeg installed on the system
- Virtual environment (recommended)

Installation Steps

1. Clone the project repository
2. Create and activate a virtual environment
3. Install dependencies using the following command:

```
pip install -r requirements.txt
```

4. Configure the NLTK offline data path
5. Maintain the required project directory structure

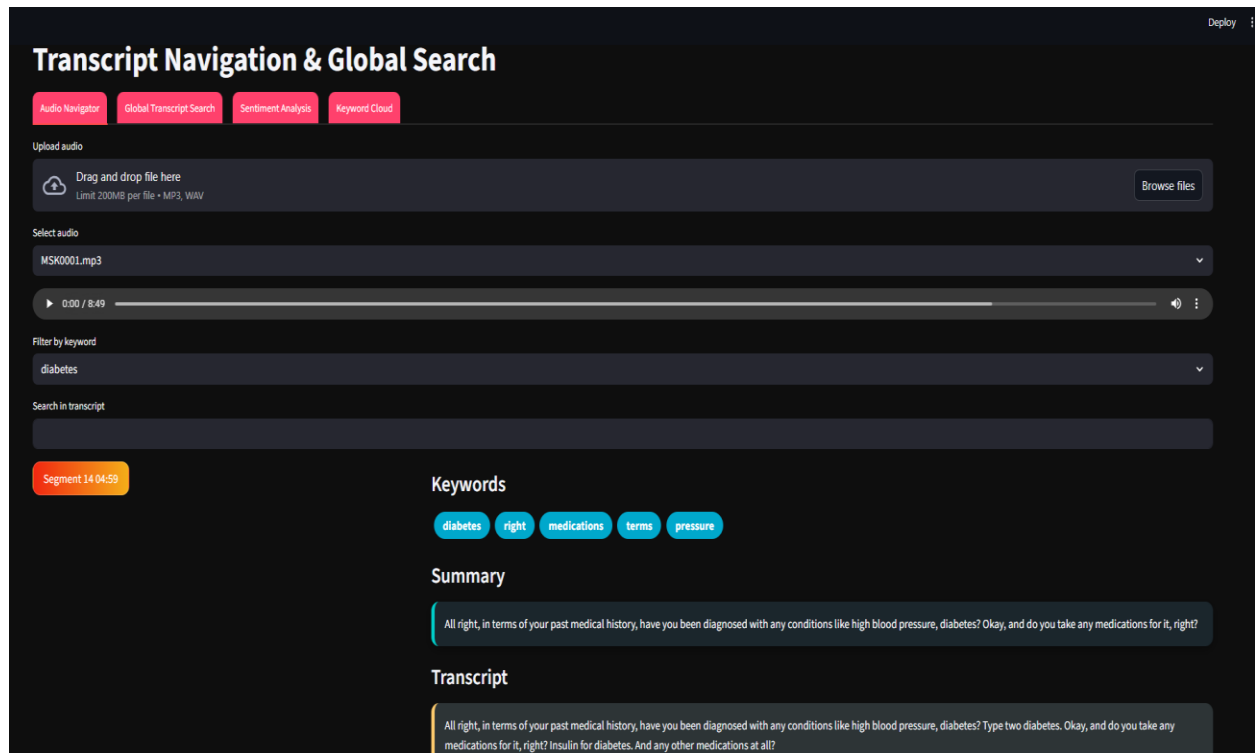
Usage

1. Place medical podcast audio files in the audio_raw directory
2. Run the audio preprocessing script
3. Execute the transcription module
4. Generate segments, keywords, and summaries
5. Launch the Streamlit application using:

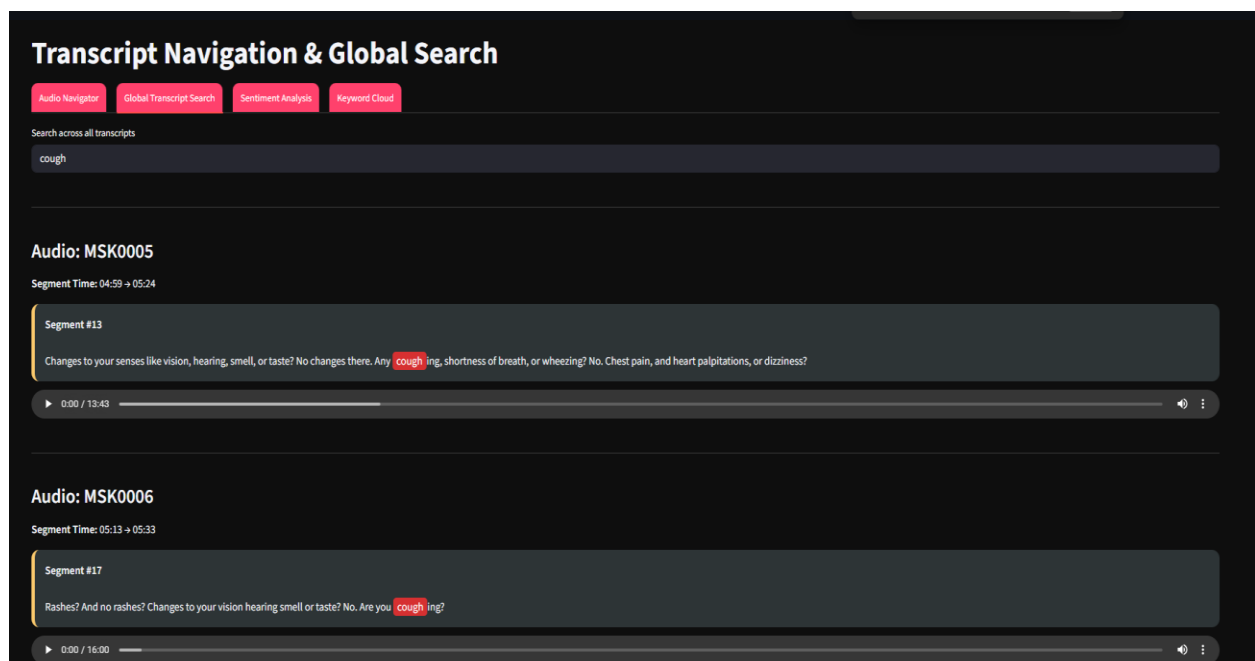
```
streamlit run app.py
```

6. View transcripts, summaries, and keyword cloud in the browser

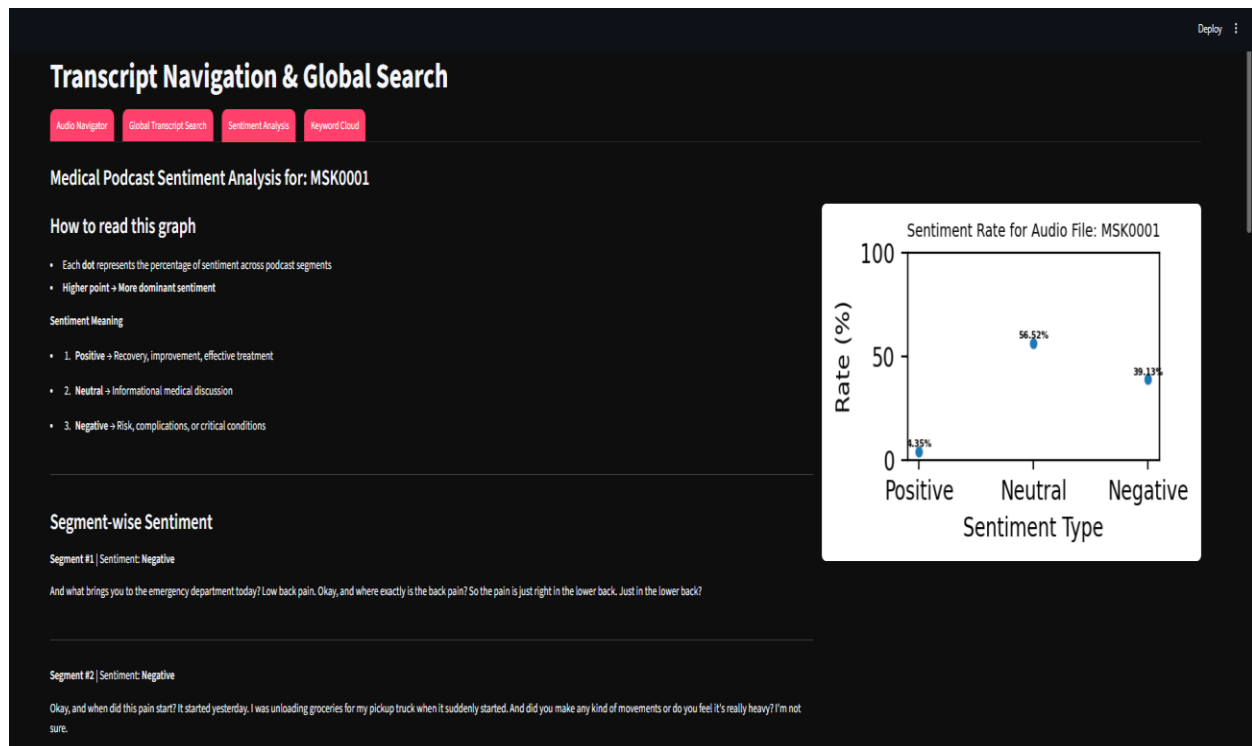
UI Preview



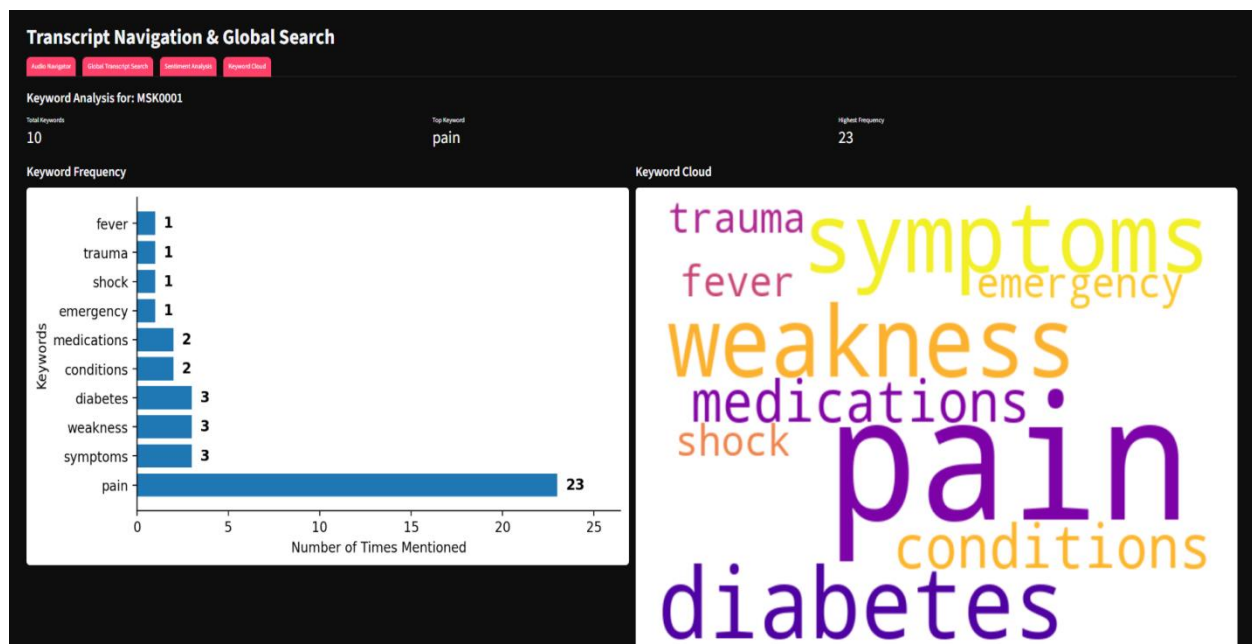
Global Search across all Transcript



Sentimental Analysis



Keyword Cloud



Troubleshooting

Issue	Possible Solution
Audio not transcribed	Verify FFmpeg installation and audio format
NLTK related errors	Check NLTK data configuration
Missing segments	Review segmentation logic
Slow processing	Use smaller Whisper models or enable GPU support

Use Cases

- Medical students reviewing podcast-based lectures and discussions.
 - Researchers analyzing expert opinions and clinical conversations.
 - Healthcare professionals summarizing long medical talks.
 - Analyzing sentiment trends in medical discussions.
 - Academic projects in Natural Language Processing and Speech Processing.
-

Limitations and Challenges

- Transcription speed depends on system performance and hardware configuration.
 - Topic segmentation is statistical and TF-IDF based, which may not fully capture semantic context.
 - Sentiment analysis uses a dictionary-based approach and may miss subtle medical sentiments.
 - Long audio files increase overall processing time.
 - Not supporting Non-English Podcast.
-

Future Enhancements

- Transformer-based topic modeling (BERTopic)
 - Speaker diarization
 - Multilingual medical podcast support
 - PDF / DOCX export of summaries
 - Medical Named Entity Recognition (NER)
 - **Advanced sentiment analysis using transformer-based models** for better context understanding
-

References

- [OpenAI Whisper Documentation](#)
- [NLTK Official Documentation](#)
- [Scikit-learn Documentation](#)
- [Streamlit Documentation](#)
- [Research papers on NLP applications in healthcare](#)